

Vapi Voicebot

Design choices

This project was designed to prioritize simplicity, modularity, and ease of deployment. The FastAPI backend handles the application logic and tools, while the static frontend provides a lightweight and fast user interface served through Nginx in Docker Compose, ensuring a clear separation between presentation and APIs. The integration with Vapi was introduced to add voice capabilities without increasing architectural complexity, and service data is stored in local JSON files to keep the system immediately usable and easy to update. Appointment management is intentionally kept in-memory using fixed slots to simulate the booking flow without requiring a database, making the prototype lightweight and straightforward to test. Docker Compose exposes the two services to mirror a typical production-like architecture while also simplifying local testing and ngrok setup for tool endpoints. Additionally, a keyword-based search approach was preferred over more advanced solutions to guarantee robustness, maintainability, and operational simplicity during the prototyping phase.

Limitations

The biggest challenge was to work with a new and unknown platform. It took time to understand how Vapi worked. One limitation is the time limit for the bot answer, it is not able to explain the entire list of services.

One clear limitation is the lack of persistent data storage in a database, due to time constraints.

I tried to create a script to perform simple scraping of the website using the BeautifulSoup library, but the result was not good enough to build a reliable RAG system. For this reason, I decided to build the prototype with hardcoded but still plausible data for the use case, in order to test the app's functionality without relying on an unreliable scraping system.

Possible improvements

The first improvement to be made is to increase the “window” that let the bot talk more in order to list all the available service. It should be an easy upgrade with a better knowledge of Vapi platform. Also, more tools could have been developed in order to help for a complete booking service.

With more time available, I would have implemented a relational database, such as PostgreSQL, orchestrated through Docker, to store the data.

Another possible improvement would be to structure a more automatic and efficient scraping system capable of extracting the information needed for the RAG features. This process requires a more in-depth study of the website and its pages, which would be feasible with more time dedicated to analysis.

AI tools used

The main AI tools used were two:

- **ChatGPT:** for general brainstorming and to speed up research on Vapi.
- **Copilot AI Agent:** an AI agent integrated into the IDE, used to develop some parts of the application more quickly, especially the frontend, and for the creation of synthetic data. Used also for debugging and better understanding of Vapi platform.